

DSC 40B

Theoretical Foundations II

Lecture 2 | Part 1

News

News

- ▶ Homework 01 and Lab 1 is posted at dsc40b.com
 - ▶ Due **Friday** @ 11:59 pm PST on Gradescope
- ▶ Office hours posted on the website
- ▶ 250+ practice problems at dsc40b.com/practice

Agenda

1. Analyzing nested loops.
2. What is Θ notation, really?

DSC 40B

Theoretical Foundations II

Lecture 2 | Part 2

Nested Loops

Example 1: Interview Problem



Example 1: Interview Problem

- ▶ Design an algorithm to solve the following problem...
- ▶ Given the heights of n people, what is the height of the tallest doctor you can make by stacking two of them?

Exercise

- ▶ What is the time complexity of the brute force solution?
- ▶ **Bonus:** what is the **best possible** time complexity of any solution?

The Brute Force Solution

- ▶ Loop through all possible (ordered) pairs.
 - ▶ How many are there?
- ▶ Check height of each.
- ▶ Keep the best.

Time/exec. # of execs.

```
def tallest_doctor(heights):  
    max_height = -float('inf')  
    n = len(heights)  
    for i in range(n):  
        for j in range(n):  
            if i == j:  
                continue  
            height = heights[i] + heights[j]  
            if height > max_height:  
                max_height = height  
    return max_height
```

Time Complexity

- ▶ Time complexity of this is $\Theta(n^2)$.
- ▶ **TODO:** Can we do better?
- ▶ Note: this algorithm considers each pair of people **twice**.
- ▶ We'll fix that in a moment.

First: A shortcut

- ▶ Making a table is getting tedious.
- ▶ Usually, find a chunk that **dominates** time complexity; i.e., yields the leading term of $T(n)$.

A Shortcut

- ▶ Assume each line takes constant time to execute *once*.
- ▶ To determine the overall time complexity:
 1. Find the line that is execute most.
 2. Count how many times it is executed.

Shortcut for the Brute Force Solution

```
for i in range(n):  
    for j in range(n):  
        height = heights[i] + heights[j] # <- count execs.
```

- ▶ On outer iter. # 1, inner body runs _____ times.
- ▶ On outer iter. # 2, inner body runs _____ times.
- ▶ On outer iter. # α , inner body runs _____ times.
- ▶ The outer loop runs _____ times.
- ▶ Total number of executions: _____

Example 2: The Median

- ▶ **Given:** real numbers $x_1, \dots, x_n$.
- ▶ **Compute:** h minimizing the **total absolute loss**

$$R(h) = \sum_{i=1} |x_i - h|$$

Example 2: The Median

- ▶ **Solution:** the **median**.
- ▶ That is, a **middle** number.
- ▶ But how do we actually **compute** a median?

A Strategy

- ▶ **Recall:** one of $x_1, \dots, x_n$ must be a median.
- ▶ **Idea:** compute $R(x_1), R(x_2), \dots, R(x_n)$, return x_i that gives the smallest result.

$$R(h) = \sum_{i=1}^n |x_i - h|$$

- ▶ Basically a **brute force** approach.

Exercise

- ▶ What is the time complexity of this brute force approach?
- ▶ How long will it take to run on an input of size 10,000?

```
def median(numbers):  
    min_h = None  
    min_value = float('inf')  
    for h in numbers:  
        total_abs_loss = 0  
        for x in numbers:  
            total_abs_loss += abs(x - h)  
        if total_abs_loss < min_value:  
            min_value = total_abs_loss  
            min_h = h  
    return min_h
```

The Median

- ▶ The brute force approach has $\Theta(n^2)$ time complexity.
- ▶ **TODO:** Is there a better algorithm?

The Median

- ▶ The brute force approach has $\Theta(n^2)$ time complexity.
- ▶ **TODO:** Is there a better algorithm?
 - ▶ It turns out, you can find the median in *linear* time.¹

¹Well, *expected* time.

```
In [8]: numbers = list(range(10_000))
```

```
In [9]: %time median(numbers)
```

```
CPU times: user 7.26 s, sys: 0 ns, total: 7.26 s
```

```
Wall time: 7.26 s
```

```
Out[9]: 4999
```

```
In [10]: %time mystery_median(numbers)
```

```
CPU times: user 4.3 ms, sys: 2 µs, total: 4.3 ms
```

```
Wall time: 4.3 ms
```

```
Out[10]: 4999
```

Careful!

- ▶ Not every nested loop has $\Theta(n^2)$ time complexity!
- ▶ In general, if:
 - ▶ outer loop iterates a times;
 - ▶ inner loop iterates b times for each outer loop iteration²;
 - ▶ then the innermost loop body is executed $a \times b$ times.

```
for x in range(n):  
    for y in range(n**2):  
        print(x + y)
```

²We are assuming here that the number of inner loop iterations doesn't depend on which outer loop iteration we're in! That is called a **dependent** nested loop.

Example 3

```
def foo(n):  
    for x in range(n):  
        for y in range(10):  
            print(x + y)
```

Example 4

```
def f(n):  
    for i in range(3*n**3 + 5*n**2 - 100):  
        for j in range(n**5, n**6):  
            print(i, j)
```


DSC 40B

Theoretical Foundations II

Lecture 2 | Part 3

Dependent Nested Loops

Example 3: Tallest Doctor, Again

- ▶ Our previous algorithm for the tallest doctor computed height for each *ordered* pair of people.
 - ▶ $i = 3$ and $j = 7$ is the same as $i = 7$ and $j = 3$
- ▶ **Idea:** consider each *unordered* pair only once:

```
for i in range(n):  
    for j in range(i + 1, n):
```

- ▶ What is the time complexity?

Pictorially

```
for i in range(4):  
    for j in range(4):  
        print(i, j)
```

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)
(3,0)	(3,1)	(3,2)	(3,3)

Pictorially

```
for i in range(4):  
    for j in range(i + 1, 4):  
        print(i, j)
```

```
(0,1) (0,2) (0,3)  
      (1,2) (1,3)  
          (2,3)
```

```
1 def tallest_doctor_2(heights):
2     max_height = -float('inf')
3     n = len(heights)
4     for i in range(n):
5         for j in range(i + 1, n):
6             height = heights[i] + heights[j]
7             if height > max_height:
8                 max_height = height
```

- ▶ **Goal:** How many times does line 6 run in total?
- ▶ Now inner nested loop **depends** on outer nested loop.

Independent

```
for i in range(n):  
    for j in range(n):  
        ...
```

- ▶ Inner loop doesn't depend on outer loop iteration #.
- ▶ Just multiply: inner body executed $n \times n = n^2$ times.

Dependent

```
for i in range(n):  
    for j in range(i, n):  
        ...
```

- ▶ Inner loop depends on outer loop iteration #.
- ▶ Can't just multiply: inner body executed ??? times.

Dependent Nested Loops

```
for i in range(n):  
    for j in range(i + 1, n):  
        height = heights[i] + heights[j]
```

- ▶ Idea: find formula $f(\alpha)$ for “number of iterations of inner loop during outer iteration α ”³

- ▶ Then total: $\sum_{\alpha=1}^n f(\alpha)$

³Why α and not i ? Python starts counting at 0, math starts at 1. Using i would be confusing – does it start at 0 or 1?

```
for i in range(n):  
    for j in range(i + 1, n):  
        height = heights[i] + heights[j]
```

- ▶ On outer iter. # 1, inner body runs _____ times.
- ▶ On outer iter. # 2, inner body runs _____ times.
- ▶ On outer iter. # α , inner body runs _____ times.
- ▶ The outer loop runs _____ times.

Totalling Up

- ▶ On outer iteration α , inner body runs $n - \alpha$ times.
 - ▶ That is, $f(\alpha) = n - \alpha$
- ▶ There are n outer iterations.
- ▶ So we need to calculate:

$$\sum_{\alpha=1}^n f(\alpha) = \sum_{\alpha=1}^n (n - \alpha)$$

$$\sum_{\alpha=1}^n (n - \alpha)$$

=

$$\underbrace{(n-1)}_{\text{1st outer iter}} + \underbrace{(n-2)}_{\text{2nd outer iter}} + \dots + \underbrace{(n-\alpha)}_{\text{kth outer iter}} + \dots + \underbrace{(n-(n-1))}_{\text{(n-1)th outer iter}} + \underbrace{(n-n)}_{\text{nth outer iter}}$$

=

$$1 + 2 + 3 + \dots + (n-3) + (n-2) + (n-1)$$

=

Aside: Arithmetic Sums

- ▶ $1 + 2 + 3 + \dots + (n-1) + n$ is an **arithmetic sum**.
- ▶ Formula for total: $n(n + 1)/2$.
- ▶ You should memorize it!

Time Complexity

- ▶ `tallest_doctor_2` has $\Theta(n^2)$ time complexity
- ▶ Same as original `tallest_doctor`!
- ▶ Should we have been able to guess this? Why?

Reason 1: Number of Pairs

- ▶ We're doing constant work for each unordered pair.
- ▶ Recall from 40A: number of pairs of n objects is

$$\binom{n}{2} = \frac{n!}{2!(n-2)!} = \frac{n(n-1)}{2}$$

- ▶ So $\Theta(n^2)$

Main Idea

If the loops are dependent, you'll usually need to write down a summation, evaluate.

Main Idea

Halving the work (or thirding, quartering, etc.) doesn't change the time complexity.

Exercise

Design a linear time algorithm for this problem.

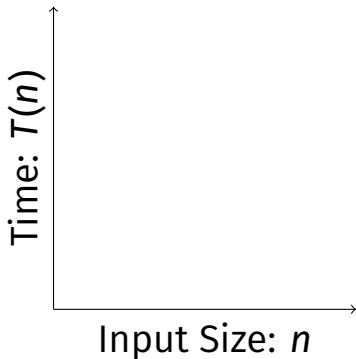
DSC 40B

Theoretical Foundations II

Lecture 2 | Part 4

Growth Rates

Linear vs. Quadratic Scaling



- ▶ $T(n) = \Theta(n)$ means " $T(n)$ grows like n "
- ▶ $T(n) = \Theta(n^2)$ means " $T(n)$ grows like n^2 "

Definition

An algorithm is said to run in **linear time** if $T(n) = \Theta(n)$.

Definition

An algorithm is said to run in **quadratic time** if $T(n) = \Theta(n^2)$.

Linear Growth

- ▶ If input size doubles, time roughly *doubles*.
- ▶ If code takes 5 seconds on 1,000 points...
- ▶ ...on 100,000 data points it takes ≈ 500 seconds.
- ▶ i.e., 8.3 minutes

Quadratic Growth

- ▶ If input size doubles, time roughly *quadruples*.
- ▶ If code takes 5 seconds on 1,000 points...
- ▶ ...on 100,000 points it takes $\approx 50,000$ seconds.
- ▶ i.e., ≈ 14 hours

In data science...

- ▶ Let's say we have a training set of 10,000 points.
- ▶ If model takes **quadratic** time to train, should expect to wait minutes to hours.
- ▶ If model takes **linear** time to train, should expect to wait seconds to minutes.
- ▶ These are rules of thumb only.

Exponential Growth

- ▶ Increasing input size by one *doubles* (triples, etc.) time taken.
- ▶ Grows very quickly!
- ▶ **Example:** brute force search of 2^n subsets.

```
for subset in all_subsets(things):  
    print(subset)
```

Logarithmic Growth

- ▶ To increase time taken by one unit, must *double* (triple, etc.) the input size.
- ▶ Grows very slowly!
- ▶ $\log n$ grows slower than n^α for *any* $\alpha > 0$
 - ▶ I.e., $\log n$ grows slower than n , $\sqrt{n}$, $n^{1/1,000}$, etc.

Exercise

What is the asymptotic time complexity of the code below as a function of n ?

```
i = 1
while i <= n:
    i = i * 2
```

Solution

- Same general strategy as before: “how many times does loop body run?”

```
i = 1
while i <= n:
    i = i * 2
```

<i>n</i>	# iters.
1	
2	
3	
4	
5	
6	
7	
8	

Common Growth Rates

- 
- ▶ $\Theta(1)$: **constant**
 - ▶ $\Theta(\log n)$: **logarithmic**
 - ▶ $\Theta(n)$: **linear**
 - ▶ $\Theta(n \log n)$: **linearithmic**
 - ▶ $\Theta(n^2)$: **quadratic**
 - ▶ $\Theta(n^3)$: **cubic**
 - ▶ $\Theta(2^n)$: **exponential**

Exercise

Which grows faster, $n!$ or 2^n ?